

```
In [2]: import numpy as np
import matplotlib.pyplot as plt
from matplotlib import colors
from sklearn.datasets import make_blobs

np.random.seed(3)
```

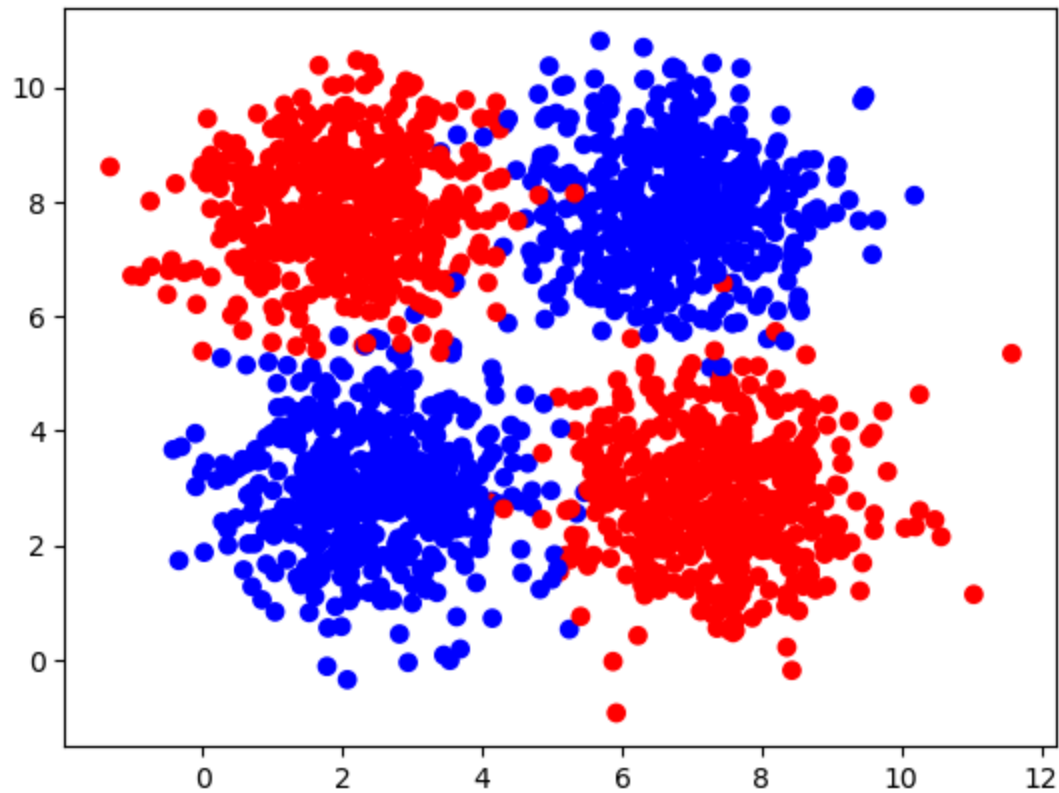
```
In [69]: m = 2000
samples, labels = make_blobs(n_samples=m,
                             centers=([2.5, 3], [6.7, 7.9], [2.1, 7.9], [7.4, 2.8]),
                             cluster_std=1.1,
                             random_state=0)

labels[(labels == 0) | (labels == 1)] = 1
labels[(labels == 2) | (labels == 3)] = 0

X = samples.T
Y = labels.reshape(1, m)

plt.scatter(X[0, :], X[1, :], c=Y, cmap=colors.ListedColormap(['red', 'blue']))
plt.show()

print(X.shape, Y.shape)
```



(2, 2000) (1, 2000)

```
In [30]: def layer_sizes(X, Y):
          n_x = X.shape[0]    # number of features = rows of X
          n_h = 2             # hidden layer size = 2 neurons
          n_y = Y.shape[0]    # number of outputs = rows of Y
          return n_x, n_h, n_y
```

```
In [31]: n_x, n_h, n_y = layer_sizes(X, Y)
          print(n_x, n_h, n_y)
```

2 2 1

```
In [6]: def layer_sizes(X,Y):
         n_x= X.shape[0]

         n_h = 2
```

```
n_y = Y.shape[0]
return (n_x , n_h , n_y)
```

```
In [7]: n_x , n_h , n_y = layer_sizes(X,Y)
```

```
In [8]: def initialize_parameters(n_x,n_h,n_y):

    W1 = np.random.randn(n_h,n_x) * 0.01 # shape(2,2) , here mutiplied by 0.01 to avoid vanishing gradient problem.

    b1 = np.zeros((n_h,1)) # shape(2,1)

    W2 = np.random.randn(n_y,n_h) * 0.01 #shape (1 ,2) , same reason as W1

    b2 = np.zeros((n_y,1)) # shape (1 ,1)

    parameters = {
        "W1":W1,
        "W2":W2,
        "b1":b1,
        "b2":b2}

    return parameters
```

```
In [9]: parameters = initialize_parameters(n_x,n_h,n_y)
```

```
In [10]: def sigmoid(z):
    res = 1/(1 + np.exp(-z))
    return res
```

```
In [11]: def forward_propagation(X , parameters):

    W1 = parameters["W1"]
    b1 = parameters["b1"]
    W2 = parameters["W2"]
    b2 = parameters["b2"]

    Z1 = np.dot(W1 , X) + b1
```

```

A1 = sigmoid(Z1)
Z2 = np.dot(W2 ,A1) + b2
A2 = sigmoid(Z2)

assert(A2.shape == (n_y , X.shape[1])) # Cheking the shape , (2,2000)

cache = {"Z1":Z1,
        "A1":A1,
        "Z2":Z2,
        "A2":A2
}

return A2 , cache

```

```
In [17]: A2 , cache = forward_propagation(X , parameters)
```

```
In [13]: def cost_function(A2,Y):
m = Y.shape[1]

logloss = (1/m) * (-Y * np.log(A2) - (1-Y) * np.log(1-A2))
cost = np.sum(logloss)

assert(isinstance(cost,float))

return cost

```

```
In [15]: cost = cost_function(A2,Y)
```

```
In [24]: def backward_propagation(parameters , cache,X , Y ):
W1 = parameters["W1"]
W2 = parameters["W2"]
A1 = cache["A1"]
A2 = cache["A2"]
m = X.shape[1]

dZ2 = A2 - Y
dW2 = 1/m * np.dot(dZ2, A1.T)
db2 = 1/m * np.sum(dZ2, axiz=1 , keepdims=True)

```

```

dZ1 = np.dot(W2.T , dZ2) * A1 * (1 - A1)
dW1 = 1/m * np.dot(dZ1 , X.T)
db1 = 1/m * np.sum(dZ1 , axis = 1 , keepdims = True)

grads = {"dW1":dW1,
         "db1":db1,
         "dW2":dW2,
         "db2":db2,
        }

return grads

```

```
In [34]: grads = backward_propagation(parameters , cache,X , Y )
```

```
In [54]: def update_paramters(parameters, grads, learning_rate=1.2):
        W1 = parameters["W1"] - 1.2 * grads["dW1"]
        b1 = parameters["b1"] - 1.2 * grads["db1"]
        W2 = parameters["W2"] - 1.2 * grads["dW2"]
        b2 = parameters["b2"] - 1.2 * grads["db2"]

        parameters = {"W1":W1,
                      "b1":b1,
                      "W2":W2,
                      "b2":b2}

        return parameters

```

```
In [55]: updated_parameters = update_paramters(parameters, grads)
```

```
In [56]: %who
```

A2	X	Y	backward_propagation	cache	colors	cost	cost_function	forward_propagation		
grads	initialize_parameters	labels	layer_sizes	m	make_blobs	n_h	n_x	n_y		
nn_model	np	parameters	plt	samples	sigmoid	update_paramters	updated_parameters			

```
In [94]: def nn_model(X , Y , n_h ,num_iterations=10 , learning_rate=1.2 , print_cost=False):
```

```
    n_x = layer_sizes(X, Y)[0]
    n_y = layer_sizes(X, Y)[2]

    parameters = initialize_parameters(n_x, n_h, n_y)

    for i in range(0,num_iterations):
        A2 , cache = forward_propagation( X , parameters)

        cost = cost_function(A2 , Y)

        grads = backward_propagation(parameters , cache , X ,Y )

        parameters = update_paramters(parameters , grads , learning_rate)

        if print_cost:
            print(f"cost after itration P {i}:{cost}")

    return parameters
```

```
In [100... parameters = nn_model(X, Y, n_h=2, num_iterations=3000, learning_rate=1.2 )
```

```
print("W1 = " + str(parameters["W1"]))
print("b1 = " + str(parameters["b1"]))
print("W2 = " + str(parameters["W2"]))
print("b2 = " + str(parameters["b2"]))
```

```
W1 = parameters["W1"]
b1 = parameters["b1"]
W2 = parameters["W2"]
b2 = parameters["b2"]
```

```
W1 = [[ 0.79804448 -0.60541652]
      [ 1.38972765 -1.1791113 ]]
b1 = [[ 0.71991726]
      [-2.76494774]]
W2 = [[ 13.08796959 -12.56869915]]
b2 = [[-3.97009079]]
```

```
In [101... def predict(X , parameters):
    A2 , cache = forward_propagation(X , parameters)
    prediction = (A2 > 0.5).astype(int)

    return prediction
```

```
In [102... X_pred = np.array([[2, 8, 2, 8], [2, 8, 8, 2]])
Y_pred = predict(X_pred, parameters)

print(f"Coordinates (in the columns):\n{X_pred}")
print(f"Predictions:\n{Y_pred}")
```

Coordinates (in the columns):

```
[[2 8 2 8]
```

```
 [2 8 8 2]]
```

Predictions:

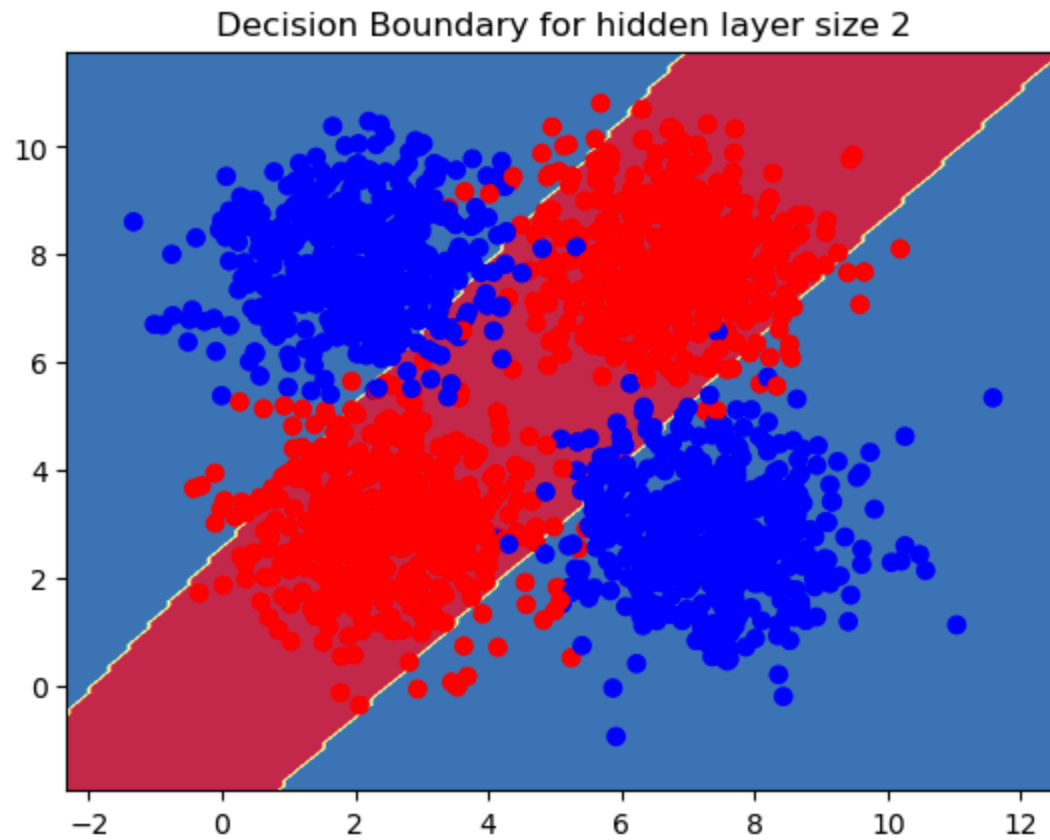
```
[[1 1 0 0]]
```

```
In [104... def plot_decision_boundary(predict, parameters, X, Y):
    # Define bounds of the domain.
    min1, max1 = X[0, :].min()-1, X[0, :].max()+1
    min2, max2 = X[1, :].min()-1, X[1, :].max()+1
    # Define the x and y scale.
    x1grid = np.arange(min1, max1, 0.1)
    x2grid = np.arange(min2, max2, 0.1)
    # Create all of the lines and rows of the grid.
    xx, yy = np.meshgrid(x1grid, x2grid)
    # Flatten each grid to a vector.
    r1, r2 = xx.flatten(), yy.flatten()
    r1, r2 = r1.reshape((1, len(r1))), r2.reshape((1, len(r2)))
    # Vertical stack vectors to create x1,x2 input for the model.
    grid = np.vstack((r1,r2))
    # Make predictions for the grid.
    predictions = predict(grid, parameters)
    # Reshape the predictions back into a grid.
    zz = predictions.reshape(xx.shape)
    # Plot the grid of x, y and z values as a surface.
    plt.contourf(xx, yy, zz, cmap=plt.cm.Spectral.reversed())
    plt.scatter(X[0, :], X[1, :], c=Y, cmap=colors.ListedColormap(['blue', 'red']));

    # Plot the decision boundary.
```

```
plot_decision_boundary(predict, parameters, X, Y)
plt.title("Decision Boundary for hidden layer size " + str(n_h))
```

Out[104... Text(0.5, 1.0, 'Decision Boundary for hidden layer size 2')



In []:

In []: